to B, the command will be:

```
user@host:/home/user$ chmod o+wx B
user@host:/home/user$ ls -l B
-rw-r--rwx  1 user user 0 Jun 14 16:15 B
```

If we want to remove group read bit for C, the command will be:

```
user@host:/home/user$ chmod g-r C
user@host:/home/user$ ls -l C
-rw----r--  1 user user 0 Jun 14 16:15 C
```

If we want to add read, write and execute bit to everyone for D, the command will be:

```
user@host:/home/user$ chmod ugo+rwx D
user@host:/home/user$ ls -l D
-rwxrwxrwx  1 user user 0 Jun 14 16:16 D
user@host:/home/user$
```

These examples should make it very clear to you as to how you can mix and match and set the desired permissions to all your files.

To use chmod with numbers, the following options can be used:

Options	Definition
#–	owner
-#-	group
–#	other
1	execute
2	write
4	reae

Owner, Group and Other are represented by three numbers. First, we determine the type of access needed for the file and then get the value for the options before adding.

Example: If you want a file to have `-rw-rw-rwx` permissions, you will use the following options:

```
user@host:/home/user$ chmod 667 XYZfile
```

If however, you want a file to have `--w-r-x--x` permissions, you will use the following:

```
user@host:/home/user$ chmod 251 ABCfile
```

Again taking the same route on files A, B, C and D, with read and write permission for the owner and read for the rest.

If we want to add owner execute bit to A, the command will be:

```
user@host:/home/user$ chmod 744 A
user@host:/home/user$ ls -l A
```